FocusForge — Backend Code

Java / Spring Boot backend files

Files included: 8

Source code reproduced from the uploaded ZIP. Each file is labeled with its path. Long lines may wrap to fit the page; code content is unchanged.

Contents

• Dockerfile
• pom.xml
• src/main/java/com/focusforge/Api.java
• src/main/java/com/focusforge/FocusForgeApplication.java
• src/main/java/com/focusforge/Models.java
• src/main/java/com/focusforge/Scheduler.java
• src/main/java/com/focusforge/Security.java
• src/main/resources/application.properties

Dockerfile

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
  </parent>
  <groupId>com.focusforge</groupId>
  <artifactId>backend</artifactId>
  <version>1.0.0</version>
  <properties><java.version>17</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependen
cy>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</artifactId></dependen
cy>
    <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>0.12.6</version></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version>0.12.6</version><scope>runtim
e</scope></dependency>
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><version>0.12.6</version><scope>run
time</scope></dependency>
  </dependencies>
  <build>
    <plugins><plugin><groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId></plugin>
</plugins>
  </build>
</project>
```

src/main/java/com/focusforge/Api.java

```
package com.focusforge;

import com.focusforge.Models.*;
import java.time.*;
import java.util.*;
import java.util.stream.Collectors;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

/** Small helpers shared by controllers. */
final class X {
    static ResponseStatusException err(int code, String msg) { return new
ResponseStatusException(HttpStatus.valueOf(code), msg); }

    static Map<String, Object> m(Object... kv) {
        Map<String, Object> map = new LinkedHashMap<>();
        for (int i = 0; i < kv.length; i += 2) map.put((String) kv[i], kv[i + 1]);
        return map;
    }

    static int streak(Collection<FocusSession> list) {
        Set<LocalDate> days = new HashSet<>();
        for (FocusSession s : list) if ("completed".equals(s.status) && s.endedAt != null)
days.add(s.endedAt.atZone(ZoneOffset.UTC).toLocalDate());
        LocalDate d = LocalDate.now(ZoneOffset.UTC);
        if (!days.contains(d)) d = d.minusDays(1);
        int n = 0;
        while (days.contains(d)) { n++; d = d.minusDays(1); }
        return n;
    }

    static int minutesSince(Collection<FocusSession> list, Instant since) {
```

```
        return list.stream().filter(s -> "completed".equals(s.status) && s.endedAt != null && s.endedAt.isAfter(since))
                .mapToInt(s -> s.plannedMinutes).sum();
    }
}
@RestController
class Health {
    @GetMapping("/") String ok() { return "FocusForge API is running"; }
}
@RestController
@RequestMapping("/api/auth")
class AuthController {
    record Creds(String email, String password, String displayName) {}
    private final UserRepo users; private final PasswordEncoder enc; private final JwtService jwt;
    AuthController(UserRepo users, PasswordEncoder enc, JwtService jwt) { this.users = users; this.enc = enc; this.jwt =
jwt; }
    @PostMapping("/register")
    Map<String, Object> register(@RequestBody Creds c) {
        if (c.email() == null || c.email().isBlank() || c.password() == null || c.password().length() < 6)
            throw X.err(400, "Enter an email and a password of at least 6 characters");
        String email = c.email().trim().toLowerCase();
        if (users.findByEmail(email).isPresent()) throw X.err(409, "That email is already registered. Log in instead");
        AppUser u = new AppUser();
        u.email = email; u.passwordHash = enc.encode(c.password());
        u.displayName = c.displayName() == null || c.displayName().isBlank() ? "Learner" : c.displayName().trim();
        users.save(u);
        return X.m("token", jwt.make(u.id), "name", u.displayName);
    }
    @PostMapping("/login")
    Map<String, Object> login(@RequestBody Creds c) {
        AppUser u = users.findByEmail(c.email() == null ? "" : c.email().trim().toLowerCase())
                .filter(x -> enc.matches(c.password() == null ? "" : c.password(), x.passwordHash))
                .orElseThrow(() -> X.err(401, "Wrong email or password"));
        return X.m("token", jwt.make(u.id), "name", u.displayName);
    }
}
@RestController
@RequestMapping("/api")
class GoalController {
    record TopicIn(String name, int difficulty, double hours) {}
    record GoalIn(String title, LocalDate deadline, double dailyHours, List<TopicIn> topics) {}
    private final GoalRepo goals; private final TopicRepo topics; private final EntryRepo entries;
    GoalController(GoalRepo goals, TopicRepo topics, EntryRepo entries) { this.goals = goals; this.topics = topics;
this.entries = entries; }
    private Goal own(Long uid, Long id) {
        Goal g = goals.findById(id).orElseThrow(() -> X.err(404, "Goal not found"));
        if (!g.userId.equals(uid)) throw X.err(403, "Not your goal");
        return g;
    }
    @PostMapping("/goals")
    Goal create(@AuthenticationPrincipal Long uid, @RequestBody GoalIn in) {
        if (in.title() == null || in.title().isBlank()) throw X.err(400, "Give your goal a name");
        if (in.deadline() == null || !in.deadline().isAfter(LocalDate.now())) throw X.err(400, "Pick a deadline after
today");
        if (in.topics() == null || in.topics().isEmpty()) throw X.err(400, "Add at least one topic");
        Goal g = new Goal();
        g.userId = uid; g.title = in.title().trim(); g.deadline = in.deadline(); g.dailyHours = in.dailyHours() <= 0 ? 2 :
 in.dailyHours();
        goals.save(g);
        List<Topic> ts = new ArrayList<>();
        for (TopicIn t : in.topics()) {
            Topic topic = new Topic();
            topic.goalId = g.id; topic.name = t.name().trim(); topic.difficulty = Math.max(1, Math.min(3,
t.difficulty())); topic.hours = Math.max(0.5, t.hours());
            ts.add(topic);
        }
        topics.saveAll(ts);
        entries.saveAll(Scheduler.build(g, ts));
        return g;
    }
    @GetMapping("/goals")
    List<Goal> list(@AuthenticationPrincipal Long uid) { return goals.findByUserId(uid); }
    @GetMapping("/goals/{id}/schedule")
    List<Entry> schedule(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        own(uid, id);
        return entries.findByGoalIdOrderByDate(id);
    }
    @DeleteMapping("/goals/{id}")
    void delete(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        Goal g = own(uid, id);
        entries.deleteAll(entries.findByGoalIdOrderByDate(id));
        topics.deleteAll(topics.findByGoalId(id));
        goals.delete(g);
    }
    @PostMapping("/schedule/{id}/toggle")
```

```
    Entry toggle(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        Entry e = entries.findById(id).orElseThrow(() -> X.err(404, "Entry not found"));
        own(uid, e.goalId);
        e.done = !e.done;
        return entries.save(e);
    }
}

@RestController
@RequestMapping("/api")
class NoteController {
    record NoteIn(String title, String content) {}
    record HlIn(int startPos, int endPos, String color) {}
    private final NoteRepo notes; private final HighlightRepo highlights;

    NoteController(NoteRepo notes, HighlightRepo highlights) { this.notes = notes; this.highlights = highlights; }

    private Note own(Long uid, Long id) {
        Note n = notes.findById(id).orElseThrow(() -> X.err(404, "Note not found"));
        if (!n.userId.equals(uid)) throw X.err(403, "Not your note");
        return n;
    }

    @GetMapping("/notes") List<Note> list(@AuthenticationPrincipal Long uid) { return
notes.findByUserIdOrderByIdDesc(uid); }

    @PostMapping("/notes")
    Note create(@AuthenticationPrincipal Long uid, @RequestBody NoteIn in) {
        Note n = new Note();
        n.userId = uid; n.title = in.title() == null || in.title().isBlank() ? "Untitled note" : in.title().trim();
        n.content = in.content() == null ? "" : in.content();
        return notes.save(n);
    }

    @PutMapping("/notes/{id}")
    Note update(@AuthenticationPrincipal Long uid, @PathVariable Long id, @RequestBody NoteIn in) {
        Note n = own(uid, id);
        String content = in.content() == null ? "" : in.content();
        if (!content.equals(n.content)) highlights.deleteAll(highlights.findByNoteId(id)); // old offsets no longer valid
        n.title = in.title() == null || in.title().isBlank() ? n.title : in.title().trim();
        n.content = content;
        return notes.save(n);
    }

    @DeleteMapping("/notes/{id}")
    void delete(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        Note n = own(uid, id);
        highlights.deleteAll(highlights.findByNoteId(id));
        notes.delete(n);
    }

    @GetMapping("/notes/{id}/highlights")
    List<Highlight> highlights(@AuthenticationPrincipal Long uid, @PathVariable Long id) { own(uid, id); return
highlights.findByNoteId(id); }

    @PostMapping("/notes/{id}/highlights")
    Highlight addHighlight(@AuthenticationPrincipal Long uid, @PathVariable Long id, @RequestBody HlIn in) {
        Note n = own(uid, id);
        if (in.startPos() < 0 || in.endPos() <= in.startPos() || in.endPos() > n.content.length()) throw X.err(400, "
Invalid selection");
        Highlight h = new Highlight();
        h.noteId = id; h.startPos = in.startPos(); h.endPos = in.endPos(); h.color = in.color() == null ? "#FFE08A" :
in.color();
        return highlights.save(h);
    }

    @DeleteMapping("/highlights/{id}")
    void deleteHighlight(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        Highlight h = highlights.findById(id).orElseThrow(() -> X.err(404, "Highlight not found"));
        own(uid, h.noteId);
        highlights.delete(h);
    }
}

/**
 * Focus sessions are validated on the server: the client must send a heartbeat at least every 90 seconds
 * and must not report leaving the tab. Only unbroken sessions that run their full length count as completed.
 */
@RestController
@RequestMapping("/api/focus")
class FocusController {
    record Start(int minutes, String technique) {}
    private static final long MAX_GAP_SECONDS = 90;
    private final SessionRepo sessions;

    FocusController(SessionRepo sessions) { this.sessions = sessions; }

    private FocusSession own(Long uid, Long id) {
        FocusSession s = sessions.findById(id).orElseThrow(() -> X.err(404, "Session not found"));
        if (!s.userId.equals(uid)) throw X.err(403, "Not your session");
        return s;
    }

    private Map<String, Object> view(FocusSession s) {
        return X.m("id", s.id, "status", s.status, "minutes", s.plannedMinutes, "technique", s.technique);
    }

    @PostMapping("/start")
```

```
    Map<String, Object> start(@AuthenticationPrincipal Long uid, @RequestBody Start in) {
        if (in.minutes() < 1 || in.minutes() > 180) throw X.err(400, "Session length must be between 1 and 180 minutes");
        for (FocusSession old : sessions.findByUserId(uid)) {
            if ("running".equals(old.status)) { old.status = "abandoned"; old.endedAt = Instant.now();
sessions.save(old); }
        }
        FocusSession s = new FocusSession();
        s.userId = uid; s.plannedMinutes = in.minutes(); s.technique = in.technique() == null ? "Pomodoro" :
in.technique();
        s.status = "running"; s.startedAt = Instant.now(); s.lastBeat = s.startedAt;
        sessions.save(s);
        return view(s);
    }
    @PostMapping("/{id}/heartbeat")
    Map<String, Object> heartbeat(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        FocusSession s = own(uid, id);
        if (!"running".equals(s.status)) return view(s);
        Instant now = Instant.now();
        if (Duration.between(s.lastBeat, now).getSeconds() > MAX_GAP_SECONDS) { s.status = "broken"; s.endedAt = now; }
        else s.lastBeat = now;
        return view(sessions.save(s));
    }
    @PostMapping("/{id}/break")
    Map<String, Object> leave(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        FocusSession s = own(uid, id);
        if ("running".equals(s.status)) { s.status = "broken"; s.endedAt = Instant.now(); sessions.save(s); }
        return view(s);
    }
    @PostMapping("/{id}/end")
    Map<String, Object> end(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        FocusSession s = own(uid, id);
        if ("running".equals(s.status)) {
            Instant now = Instant.now();
            long elapsed = Duration.between(s.startedAt, now).getSeconds();
            long gap = Duration.between(s.lastBeat, now).getSeconds();
            s.status = gap > MAX_GAP_SECONDS ? "broken" : elapsed >= s.plannedMinutes * 60L - 5 ? "completed" : "
abandoned";
            s.endedAt = now;
            sessions.save(s);
        }
        return view(s);
    }
    @GetMapping("/stats")
    Map<String, Object> stats(@AuthenticationPrincipal Long uid) {
        List<FocusSession> all = sessions.findByUserId(uid);
        long completed = all.stream().filter(s -> "completed".equals(s.status)).count();
        long broken = all.stream().filter(s -> "broken".equals(s.status)).count();
        List<Map<String, Object>> week = new ArrayList<>();
        for (int i = 6; i >= 0; i--) {
            LocalDate d = LocalDate.now(ZoneOffset.UTC).minusDays(i);
            int mins = all.stream().filter(s -> "completed".equals(s.status) && s.endedAt != null &&
s.endedAt.atZone(ZoneOffset.UTC).toLocalDate().equals(d))
                    .mapToInt(s -> s.plannedMinutes).sum();
            week.add(X.m("day", d.getDayOfWeek().toString().substring(0, 3), "minutes", mins));
        }
        return X.m("totalMinutes", X.minutesSince(all, Instant.EPOCH), "completed", completed, "broken", broken, "
streak", X.streak(all), "week", week);
    }
}
/** Rooms: members only ever see anonymous handles. No chat endpoint exists on purpose. */
@RestController
@RequestMapping("/api/rooms")
class RoomController {
    record RoomIn(String name, String category) {}
    record ShareIn(String title, String content, String link) {}
    private static final String[] ADJ = {"Focused", "Calm", "Bright", "Steady", "Swift", "Quiet", "Bold", "Keen"};
    private static final String[] ANIMAL = {"Falcon", "Otter", "Maple", "Comet", "Panda", "Cedar", "Lynx", "Heron"};
    private final RoomRepo rooms; private final MemberRepo members; private final SharedRepo shared; private final
SessionRepo sessions;
    RoomController(RoomRepo rooms, MemberRepo members, SharedRepo shared, SessionRepo sessions) {
        this.rooms = rooms; this.members = members; this.shared = shared; this.sessions = sessions;
    }
    private Member join(Long roomId, Long uid) {
        return members.findByRoomIdAndUserId(roomId, uid).orElseGet(() -> {
            Set<String> taken = members.findByRoomId(roomId).stream().map(m -> m.handle).collect(Collectors.toSet());
            Random r = new Random();
            String handle;
            do { handle = ADJ[r.nextInt(ADJ.length)] + " " + ANIMAL[r.nextInt(ANIMAL.length)] + " " + (10 +
r.nextInt(90)); } while (taken.contains(handle));
            Member m = new Member();
            m.roomId = roomId; m.userId = uid; m.handle = handle;
            return members.save(m);
        });
    }
    @GetMapping
```

```
    List<Map<String, Object>> list(@AuthenticationPrincipal Long uid) {
        Set<Long> mine = members.findByUserId(uid).stream().map(m -> m.roomId).collect(Collectors.toSet());
        return rooms.findAll().stream().map(r -> X.m("id", r.id, "name", r.name, "category", r.category,
                "members", members.findByRoomId(r.id).size(), "joined", mine.contains(r.id))).toList();
    }

    @PostMapping
    Map<String, Object> create(@AuthenticationPrincipal Long uid, @RequestBody RoomIn in) {
        if (in.name() == null || in.name().isBlank()) throw X.err(400, "Give the room a name");
        Room r = new Room();
        r.name = in.name().trim(); r.category = in.category() == null || in.category().isBlank() ? "General" :
in.category().trim(); r.createdBy = uid;
        rooms.save(r);
        Member m = join(r.id, uid);
        return X.m("id", r.id, "handle", m.handle);
    }

    @PostMapping("/{id}/join")
    Map<String, Object> joinRoom(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        rooms.findById(id).orElseThrow(() -> X.err(404, "Room not found"));
        return X.m("handle", join(id, uid).handle);
    }

    @GetMapping("/{id}")
    Map<String, Object> view(@AuthenticationPrincipal Long uid, @PathVariable Long id) {
        Room room = rooms.findById(id).orElseThrow(() -> X.err(404, "Room not found"));
        Member me = members.findByRoomIdAndUserId(id, uid).orElseThrow(() -> X.err(403, "Join the room first"));
        List<Member> all = members.findByRoomId(id);
        Map<Long, String> handles = all.stream().collect(Collectors.toMap(m -> m.id, m -> m.handle));
        Instant weekAgo = Instant.now().minus(Duration.ofDays(7));
        List<Map<String, Object>> people = new ArrayList<>();
        for (Member m : all) {
            List<FocusSession> ss = sessions.findByUserId(m.userId);
            people.add(X.m("handle", m.handle, "weekMinutes", X.minutesSince(ss, weekAgo), "streak", X.streak(ss), "me",
m.id.equals(me.id)));
        }
        people.sort((a, b) -> Integer.compare((int) b.get("weekMinutes"), (int) a.get("weekMinutes")));
        List<Map<String, Object>> items = shared.findByRoomIdOrderBySharedAtDesc(id).stream().map(s -> X.m(
                "id", s.id, "by", handles.getOrDefault(s.memberId, "Former member"), "title", s.title,
                "content", s.content == null ? "" : s.content, "link", s.link == null ? "" : s.link,
                "at", s.sharedAt.toString(), "mine", s.memberId.equals(me.id))).toList();
        return X.m("name", room.name, "category", room.category, "myHandle", me.handle, "people", people, "items", items);
    }

    @PostMapping("/{id}/share")
    Map<String, Object> share(@AuthenticationPrincipal Long uid, @PathVariable Long id, @RequestBody ShareIn in) {
        Member me = members.findByRoomIdAndUserId(id, uid).orElseThrow(() -> X.err(403, "Join the room first"));
        if (in.title() == null || in.title().isBlank()) throw X.err(400, "Give the material a title");
        boolean noContent = in.content() == null || in.content().isBlank();
        boolean noLink = in.link() == null || in.link().isBlank();
        if (noContent && noLink) throw X.err(400, "Add some text or a link to share");
        Shared s = new Shared();
        s.roomId = id; s.memberId = me.id; s.title = in.title().trim();
        s.content = noContent ? null : in.content().substring(0, Math.min(20000, in.content().length()));
        s.link = noLink ? null : in.link().trim();
        s.sharedAt = Instant.now();
        shared.save(s);
        return X.m("id", s.id);
    }

    @DeleteMapping("/{id}/items/{itemId}")
    void removeItem(@AuthenticationPrincipal Long uid, @PathVariable Long id, @PathVariable Long itemId) {
        Member me = members.findByRoomIdAndUserId(id, uid).orElseThrow(() -> X.err(403, "Join the room first"));
        Shared s = shared.findById(itemId).orElseThrow(() -> X.err(404, "Item not found"));
        if (!s.memberId.equals(me.id)) throw X.err(403, "You can only remove your own items");
        shared.delete(s);
    }
}
```

src/main/java/com/focusforge/FocusForgeApplication.java

```
package com.focusforge;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class FocusForgeApplication {
    public static void main(String[] args) {
        SpringApplication.run(FocusForgeApplication.class, args);
    }
}
```

src/main/java/com/focusforge/Models.java

```
package com.focusforge;

import jakarta.persistence.*;
```

```
import java.time.Instant;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

/** All entities and repositories. Public fields keep the code short (Hibernate field access + Jackson). */
public final class Models {
    private Models() {}

    @Entity @Table(name = "app_users")
    public static class AppUser {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        @Column(unique = true) public String email;
        public String passwordHash;
        public String displayName;
    }

    @Entity @Table(name = "goals")
    public static class Goal {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long userId;
        public String title;
        public LocalDate deadline;
        public double dailyHours;
    }

    @Entity @Table(name = "topics")
    public static class Topic {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long goalId;
        public String name;
        public int difficulty; // 1 easy, 2 medium, 3 hard
        public double hours;
    }

    /** One planned study block. type = first_pass | revision_1 | revision_2 | revision_3 */
    @Entity @Table(name = "schedule_entries")
    public static class Entry {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long goalId;
        public String topic;
        public LocalDate date;
        public String type;
        public int minutes;
        public boolean done;
    }

    @Entity @Table(name = "notes")
    public static class Note {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long userId;
        public String title;
        @Column(length = 100000) public String content;
    }

    @Entity @Table(name = "highlights")
    public static class Highlight {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long noteId;
        public int startPos;
        public int endPos;
        public String color;
    }

    @Entity @Table(name = "focus_sessions")
    public static class FocusSession {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long userId;
        public int plannedMinutes;
        public String technique;
        public String status; // running | completed | broken | abandoned
        public Instant startedAt;
        public Instant lastBeat;
        public Instant endedAt;
    }

    @Entity @Table(name = "rooms")
    public static class Room {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public String name;
        public String category;
        public Long createdBy;
    }

    /** Links a real user to a room under an anonymous handle. userId is NEVER sent to other members. */
    @Entity @Table(name = "room_members")
    public static class Member {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long roomId;
        public Long userId;
        public String handle;
    }

    @Entity @Table(name = "shared_items")
```

```
    public static class Shared {
        @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
        public Long roomId;
        public Long memberId;
        public String title;
        @Column(length = 20000) public String content;
        @Column(length = 2000) public String link;
        public Instant sharedAt;
    }
    public interface UserRepo extends JpaRepository<AppUser, Long> { Optional<AppUser> findByEmail(String email); }
    public interface GoalRepo extends JpaRepository<Goal, Long> { List<Goal> findByUserId(Long userId); }
    public interface TopicRepo extends JpaRepository<Topic, Long> { List<Topic> findByGoalId(Long goalId); }
    public interface EntryRepo extends JpaRepository<Entry, Long> { List<Entry> findByGoalIdOrderByDate(Long goalId); }
    public interface NoteRepo extends JpaRepository<Note, Long> { List<Note> findByUserIdOrderByIdDesc(Long userId); }
    public interface HighlightRepo extends JpaRepository<Highlight, Long> { List<Highlight> findByNoteId(Long noteId); }
    public interface SessionRepo extends JpaRepository<FocusSession, Long> { List<FocusSession> findByUserId(Long
userId); }
    public interface RoomRepo extends JpaRepository<Room, Long> {}
    public interface MemberRepo extends JpaRepository<Member, Long> {
        List<Member> findByRoomId(Long roomId);
        List<Member> findByUserId(Long userId);
        Optional<Member> findByRoomIdAndUserId(Long roomId, Long userId);
    }
    public interface SharedRepo extends JpaRepository<Shared, Long> { List<Shared> findByRoomIdOrderBySharedAtDesc(Long
roomId); }
}
```

src/main/java/com/focusforge/Scheduler.java

```
package com.focusforge;

import com.focusforge.Models.*;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;

/**
 * Study plan generator.
 * 1. First-pass window = first 70% of the days until the deadline (the rest is reserved for revision).
 * 2. Each topic needs hours * difficulty multiplier; if that exceeds the window, all topics are scaled down evenly.
 * 3. Hardest topics are placed first, filling each day up to the daily limit.
 * 4. Spaced-repetition revisions are added at +2, +6 and +14 days after each topic's last first-pass day,
 *    skipping any that fall after the deadline.
 */
final class Scheduler {
    private static final int[] REVISION_GAPS = {2, 6, 14};

    static List<Entry> build(Goal goal, List<Topic> topics) {
        LocalDate today = LocalDate.now();
        long days = ChronoUnit.DAYS.between(today, goal.deadline) + 1;
        int dailyMin = (int) Math.max(30, Math.round(goal.dailyHours * 60));
        long window = Math.max(1, (long) Math.floor(days * 0.7));

        List<Topic> sorted = new ArrayList<>(topics);
        sorted.sort(Comparator.comparingInt((Topic t) -> t.difficulty).reversed());

        double need = 0;
        for (Topic t : sorted) need += weighted(t);
        double scale = need == 0 ? 1 : Math.min(1.0, (window * (double) dailyMin) / need);

        List<Entry> out = new ArrayList<>();
        LocalDate day = today;
        int used = 0;
        for (Topic t : sorted) {
            int total = Math.max(15, (int) Math.round(weighted(t) * scale));
            int remaining = total;
            LocalDate last = day;
            while (remaining > 0) {
                if (used >= dailyMin) { day = day.plusDays(1); used = 0; }
                int chunk = Math.min(remaining, dailyMin - used);
                out.add(entry(goal, t.name, day, "first_pass", chunk));
                used += chunk; remaining -= chunk; last = day;
            }
            for (int i = 0; i < REVISION_GAPS.length; i++) {
                LocalDate d = last.plusDays(REVISION_GAPS[i]);
                if (!d.isAfter(goal.deadline)) out.add(entry(goal, t.name, d, "revision_" + (i + 1), Math.max(15, total /
5)));
            }
        }
        out.sort(Comparator.comparing((Entry e) -> e.date).thenComparing(e -> e.type));
        return out;
    }
    private static double weighted(Topic t) {
        double mult = t.difficulty >= 3 ? 1.3 : t.difficulty == 1 ? 0.8 : 1.0;
        return t.hours * 60 * mult;
    }
    private static Entry entry(Goal g, String topic, LocalDate date, String type, int minutes) {
        Entry e = new Entry();
```

```
        e.goalId = g.id; e.topic = topic; e.date = date; e.type = type; e.minutes = minutes;
        return e;
    }
}
```

src/main/java/com/focusforge/Security.java

```
package com.focusforge;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Date;
import java.util.List;
import javax.crypto.SecretKey;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.stereotype.Component;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
class JwtService {
    private final SecretKey key;

    JwtService(@Value("${app.jwt.secret}") String secret) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    String make(Long userId) {
        return Jwts.builder().subject(String.valueOf(userId)).issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + 7L * 24 * 3600 * 1000))
                .signWith(key).compact();
    }

    Long parse(String token) {
        return Long.valueOf(Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload().getSubject());
    }
}

@Component
class JwtFilter extends OncePerRequestFilter {
    private final JwtService jwt;

    JwtFilter(JwtService jwt) { this.jwt = jwt; }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String h = req.getHeader("Authorization");
        if (h != null && h.startsWith("Bearer ")) {
            try {
                Long id = jwt.parse(h.substring(7));
                SecurityContextHolder.getContext().setAuthentication(new UsernamePasswordAuthenticationToken(id, null,
List.of()));
            } catch (Exception ignored) { /* invalid token -> stays unauthenticated */ }
        }
        chain.doFilter(req, res);
    }
}

@Configuration
class SecurityConfig {
    @Bean
    SecurityFilterChain chain(HttpSecurity http, JwtFilter filter, @Value("${app.frontend-url}") String frontend) throws
Exception {
        CorsConfiguration cors = new CorsConfiguration();
        cors.setAllowedOrigins(Arrays.stream(frontend.split(",")).map(String::trim).toList());
        cors.setAllowedMethods(List.of("*"));
        cors.setAllowedHeaders(List.of("*"));
        UrlBasedCorsConfigurationSource src = new UrlBasedCorsConfigurationSource();
        src.registerCorsConfiguration("/**", cors);

        http.csrf(c -> c.disable())
            .cors(c -> c.configurationSource(src))
```

```
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(a -> a
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                .requestMatchers("/", "/error", "/api/auth/**").permitAll()
                .anyRequest().authenticated())
            .exceptionHandling(e -> e.authenticationEntryPoint((rq, rs, ex) -> rs.sendError(401)))
            .addFilterBefore(filter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
    @Bean
    PasswordEncoder encoder() { return new BCryptPasswordEncoder(); }
}
```

src/main/resources/application.properties

```
server.port=${PORT:8080}
server.error.include-message=always
spring.datasource.url=${DB_URL:jdbc:h2:mem:focusforge;DB_CLOSE_DELAY=-1}
spring.datasource.username=${DB_USER:sa}
spring.datasource.password=${DB_PASS:}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
app.jwt.secret=${JWT_SECRET:change-this-secret-change-this-secret-123456}
app.frontend-url=${FRONTEND_URL:http://localhost:5173}
```